

Universidad Nacional de San Agustín



Facultad de Ingeniería de Producción y Servicios

Escuela Profesional de Ingeniería de Sistemas

PROGRAMACIÓN DE SISTEMAS

DOCUMENTACIÓN TÉCNICA

Proyecto Final: Programa ADMIN en Linux

Docente:

Norman Patrick Harvey Arce

Integrantes:

Condori Catunta, Joselin Sharon

Rojas Condori, Fabiana Paola

Schreiber Landeo, Diego Hans

2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Equipos, materiales y temas utilizados	3
4. Arquitectura del Sistema	4
5. Estructura del Proyecto	4
6. Descripción de los módulos	6
6.1. Administrador de tareas	6
6.2. Shell de archivos	6
6.3. Ejecución de comandos Linux	6
6.4. Sistema de respaldos	7
6.5. Analizador de scripts Bash	7
6.6. Cola de descargas	7
6.7. Información del Sistema	7
7. Flujo general del sistema	8
7.1. Diagrama de flujo general	8
8. Llamadas al sistema y funciones utilizadas	9
8.1. Administración de procesos	9
8.2. Administración de archivos	9
8.3. Ejecución de comandos	10
8.4. Entrada y salida	10
8.5. Manejo de directorios	10
8.6. Resumen	10
9. Compilación e instalación del proyecto	10
9.1. Requisitos del sistema	11
9.2. Estructura del proyecto	11
9.3. Proceso de compilación	11
9.4. Limpieza del proyecto	11
9.5. Ejecución del programa	12
10. Pruebas realizadas	12
10.1. Pruebas del administrador de tareas	12
10.2. Pruebas del shell de archivos	15
10.3. Pruebas de ejecución de comandos	18
10.4. Pruebas del sistema de respaldos	19
10.5. Pruebas del analizador Bash	19
10.6. Pruebas de la cola de descargas	19
10.7. Pruebas del módulo de Información del Sistema	20
10.8. Resumen de pruebas	20

11.Conclusiones y trabajos futuros	20
11.1. Conclusiones	20
11.2. Trabajos futuros	21

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación de consola en lenguaje C para el sistema operativo Linux, que proporciona herramientas de administración del sistema de forma modular e interactiva. El sistema permite gestionar procesos, archivos, comandos personalizados, respaldos, análisis de scripts Bash y colas de descargas.

El propósito técnico es consolidar los conocimientos de programación de sistemas, haciendo uso intensivo de llamadas al sistema (system calls) como `fork()`, `exec()`, `wait()`, `kill()`, `opendir()`, `readdir()`, entre otras, así como el manejo de señales, tuberías y procesos en entorno Unix. La aplicación está estructurada de manera que cada módulo es independiente y puede ser desarrollado y probado por separado, facilitando el trabajo colaborativo mediante Git.

2. Objetivos

2.1. Objetivo General

Desarrollar una herramienta de administración de sistemas Linux en C que integre funcionalidades esenciales para la gestión de procesos, archivos, comandos, respaldos, análisis de scripts y descargas, aplicando los conceptos de programación de sistemas y buenas prácticas de desarrollo.

2.2. Objetivos Específicos

- Implementar un administrador de procesos que permita listar, buscar, finalizar, suspender y reanudar procesos, así como mostrar el árbol de procesos y el consumo de recursos.
- Desarrollar un shell de archivos con operaciones básicas (listar, copiar, mover, eliminar) usando llamadas al sistema.
- Crear un ejecutor de comandos Linux que permita al usuario introducir cualquier comando, mostrar su salida y errores, y guardar un historial.
- Diseñar un sistema de respaldos que comprima y almacene copias de seguridad de directorios.
- Construir un analizador sintáctico para scripts Bash que valide la estructura de los comandos.
- Gestionar una cola de descargas de archivos desde URLs.
- Organizar el proyecto de forma modular con Makefile, documentación y control de versiones.

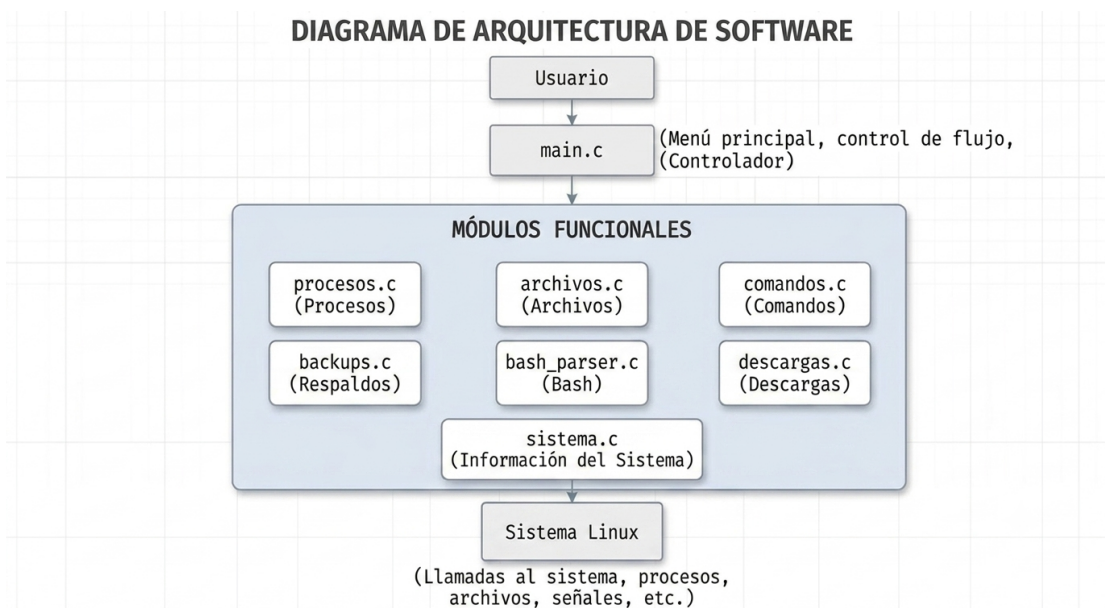
3. Equipos, materiales y temas utilizados

- **Lenguaje C (estándar GNU99)** – Lenguaje de programación principal utilizado para el desarrollo del proyecto, seleccionado por su eficiencia y compatibilidad nativa con las llamadas al sistema operativo.
- **Sistema Operativo Linux Ubuntu 22.04 LTS** – Entorno de ejecución e infraestructura base sobre la cual se despliega y prueba el administrador del sistema.
- **Compilador GCC (v11+)** – Colección de compiladores de GNU utilizada para transformar el código fuente en C en el ejecutable binario final del sistema.
- **Librerías estándar de C** – Cabeceras esenciales (`stdio.h`, `stdlib.h`, `string.h`, `unistd.h`, `signal.h`, `sys/types.h`, `sys/wait.h`, `dirent.h`, `sys/stat.h`, `time.h`) utilizadas para la gestión de entrada/salida, manejo de memoria, control de procesos y manipulación de archivos.

- **Llamadas al sistema (System Calls)** – Funciones del núcleo de Linux (`fork()`, `execvp()`, `execlp()`, `wait()`, `waitpid()`, `kill()`, `opendir()`, `readdir()`, `stat()`) empleadas para interactuar directamente con el sistema operativo a bajo nivel.
- **Make** – Herramienta de automatización utilizada para gestionar la compilación eficiente del proyecto mediante un archivo `Makefile`.
- **Git** – Sistema de control de versiones descentralizado empleado para registrar el historial de cambios y coordinar el desarrollo del código.
- **GitHub** – Plataforma de alojamiento en la nube utilizada para almacenar el repositorio remoto y facilitar el trabajo colaborativo.
- **LaTeX** – Lenguaje de marcado utilizado para la redacción formal de este informe.
- **Overleaf** – Plataforma online colaborativa usada para redactar el informe técnico en LaTeX.

4. Arquitectura del Sistema

La arquitectura del sistema es de tipo monolítica modular, donde el punto de entrada es `main.c`, el cual controla el menú principal y deriva la ejecución a los módulos especializados.



Cada módulo tiene su propio submenú y funciones independientes, lo que permite que diferentes integrantes trabajen en paralelo sin conflictos. La comunicación entre módulos se realiza únicamente a través de `main.c`, que invoca las funciones de menú de cada módulo.

5. Estructura del Proyecto

El árbol de directorios del proyecto es el siguiente:

```

Proyecto-Admin-Linux
|--- Makefile           # Script de compilacion
|--- README.md          # Descripcion general
|--- .gitignore         # Archivos ignorados por Git
|

```

```

|--- src                                # Código fuente
|   |--- archivos.c
|   |--- backups.c
|   |--- bash_parser.c
|   |--- comandos.c
|   |--- descargas.c
|   |--- main.c
|   |--- main.o
|   |--- procesos.c
|   |--- sistema.c
|
|--- include                            # Archivos de cabecera
|   |--- archivos.h
|   |--- backups.h
|   |--- bash_parser.h
|   |--- comandos.h
|   |--- descargas.h
|   |--- procesos.h
|   |--- sistema.h
|
|--- backups                           # Directorio copias de seguridad
|
|--- scripts                           # Scripts Bash prueba para analizador
|   |--- prueba2.sh
|   |--- prueba3.sh
|
|--- historial                         # Historial de comandos ejecutados
|   |--- historial.txt
|
|--- downloads                         # Archivos descargados
|
|--- docs                             # Documentación
|   |--- Manual_Usuario.pdf
|   |--- Documentacion_Tecnica.pdf
|   |--- Presentacion.pptx
|
|--- bin                              # Ejecutable compilado
|   |--- admin_linux
|

```

Descripción de cada carpeta:

- **src/**: Contiene todos los archivos .c con la implementación de cada módulo y el main.
- **include/**: Archivos .h con los prototipos de funciones y definiciones compartidas.
- **backups/**: Espacio de almacenamiento para los respaldos generados por el módulo de respaldos.
- **scripts/**: Scripts de prueba en Bash para evaluar el analizador sintáctico.
- **historial/**: Archivo de texto donde se registran los comandos ejecutados con fecha y hora.
- **descargas/**: Destino de los archivos descargados mediante el módulo de descargas.
- **docs/**: Documentación del proyecto en formato PDF y presentación.
- **bin/**: Ejecutable final del proyecto, generado por el Makefile.

6. Descripción de los módulos

El proyecto fue desarrollado utilizando una arquitectura modular, donde cada componente implementa una funcionalidad específica de la herramienta de administración para Linux. Esta organización facilita el mantenimiento, la reutilización del código y la incorporación de nuevas funcionalidades sin afectar el resto del sistema.

6.1. Administrador de tareas

Este módulo permite supervisar y administrar los procesos activos del sistema operativo. Sus principales funciones son:

- Listado de procesos en ejecución.
- Búsqueda de procesos por nombre.
- Visualización del consumo de CPU y memoria.
- Finalización de procesos mediante señales del sistema.
- Suspensión y reanudación de procesos.
- Visualización del árbol de procesos para identificar relaciones padre-hijo.

La implementación utiliza comandos del sistema y llamadas al sistema de Linux para interactuar con los procesos existentes.

6.2. Shell de archivos

Este módulo proporciona una interfaz sencilla para la administración de archivos y directorios. Entre sus funciones se encuentran:

- Listar archivos de un directorio.
- Copiar archivos.
- Mover archivos.
- Eliminar archivos.
- Buscar archivos por nombre.
- Mostrar estadísticas básicas de archivos y directorios.

Estas operaciones permiten administrar el sistema de archivos sin abandonar la aplicación.

6.3. Ejecución de comandos Linux

Este componente permite ejecutar comandos directamente desde la herramienta. Sus características principales son:

- Recepción del comando ingresado por el usuario.
- Ejecución utilizando el intérprete de comandos.
- Visualización de la salida estándar.
- Registro de los comandos ejecutados en un historial.

El historial permite consultar posteriormente las operaciones realizadas durante la sesión.

6.4. Sistema de respaldos

El módulo de respaldos permite generar copias de seguridad de archivos y directorios importantes. Entre las funciones implementadas destacan:

- Creación de respaldos.
- Almacenamiento organizado de versiones.
- Restauración de respaldos previamente generados.
- Consulta del historial de respaldos disponibles.

Esta funcionalidad reduce el riesgo de pérdida de información durante la administración del sistema.

6.5. Analizador de scripts Bash

Este módulo analiza archivos escritos en Bash con el objetivo de identificar algunos de sus componentes principales.

Actualmente permite detectar:

- Variables declaradas.
- Estructuras repetitivas (for y while).
- Información básica del script analizado.

El análisis se realiza leyendo el archivo línea por línea e identificando patrones característicos del lenguaje Bash.

6.6. Cola de descargas

Este módulo administra una lista de descargas simuladas y registra los eventos asociados.

Entre sus funciones se encuentran:

- Agregar nuevas descargas.
- Mostrar la cola de descargas.
- Registrar eventos durante la ejecución.
- Mostrar el progreso de las tareas.

Este componente demuestra el manejo organizado de tareas pendientes dentro de la aplicación.

6.7. Información del Sistema

Este módulo permite obtener información relevante del sistema operativo mediante la ejecución de comandos Linux. Para ello, utiliza llamadas al sistema como `fork()`, `exec()` y `wait()`, permitiendo crear procesos independientes para consultar los datos del equipo.

Entre sus funciones se encuentran:

- Mostrar información del sistema operativo.
- Consultar características del procesador.
- Mostrar el estado y uso de la memoria RAM.
- Consultar el espacio disponible en disco.
- Mostrar el tiempo de actividad del sistema.
- Mostrar el usuario actual conectado.

7. Flujo general del sistema

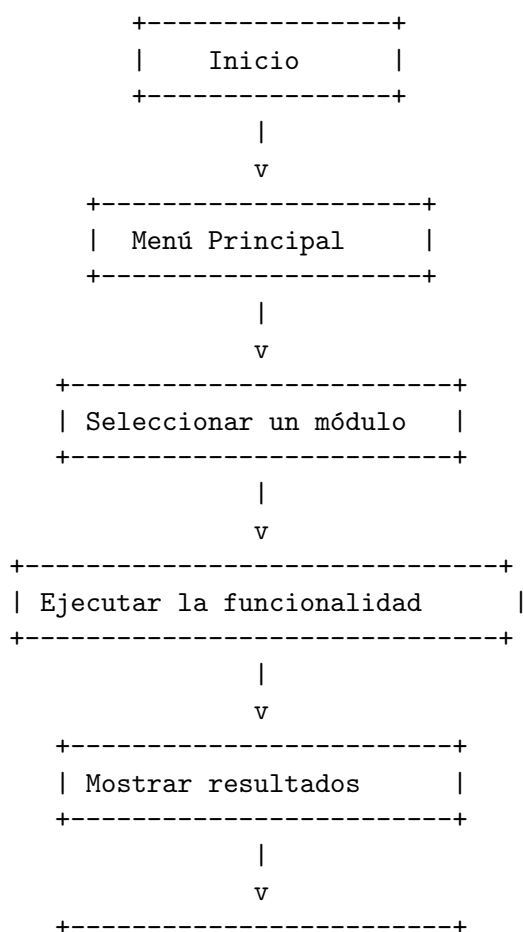
La herramienta fue diseñada siguiendo un flujo de ejecución basado en un menú principal desde el cual el usuario puede acceder a los diferentes módulos del sistema. Cada opción seleccionada invoca una función específica encargada de realizar la operación correspondiente y, al finalizar, retorna nuevamente al menú principal.

El flujo de ejecución general puede resumirse de la siguiente manera:

1. Inicio de la aplicación.
2. Carga del menú principal.
3. Selección de una opción por parte del usuario.
4. Ejecución del módulo correspondiente.
5. Procesamiento de la solicitud.
6. Presentación de los resultados.
7. Retorno al menú principal.
8. Finalización del programa cuando el usuario selecciona la opción de salir.

Este diseño permite mantener una interacción sencilla con el usuario y facilita la incorporación de nuevos módulos sin modificar la estructura principal del programa.

7.1. Diagrama de flujo general



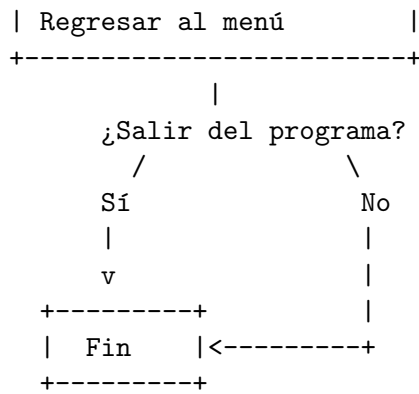


Figura 2. Flujo general de ejecución de la herramienta.

8. Llamadas al sistema y funciones utilizadas

El desarrollo de la herramienta aprovecha diversas funciones de la biblioteca estándar de C y comandos propios del sistema operativo Linux para implementar las funcionalidades solicitadas. Estas funciones permiten interactuar con el sistema operativo, el sistema de archivos y los procesos en ejecución.

8.1. Administración de procesos

Para la gestión de procesos se utilizan comandos propios de Linux invocados desde el programa, permitiendo obtener información y controlar procesos existentes.

Entre las operaciones realizadas se encuentran:

- Consulta de procesos activos.
- Visualización del consumo de CPU y memoria.
- Finalización de procesos.
- Suspensión y reanudación mediante señales del sistema.
- Visualización del árbol de procesos.

Estas funcionalidades permiten supervisar el estado del sistema desde una única interfaz.

8.2. Administración de archivos

Las operaciones sobre archivos utilizan funciones de la biblioteca estándar y comandos del sistema para manipular directorios y archivos.

Las principales operaciones implementadas son:

- Apertura y lectura de directorios.
- Copia de archivos.
- Movimiento de archivos.
- Eliminación de archivos.
- Búsqueda por nombre.
- Obtención de información básica de los archivos.

Estas operaciones permiten administrar el sistema de archivos de manera sencilla desde la aplicación.

8.3. Ejecución de comandos

La herramienta permite ejecutar comandos del sistema operativo ingresados por el usuario. Para ello se emplean funciones de ejecución de comandos que permiten:

- Enviar el comando al sistema operativo.
- Mostrar la salida generada.
- Registrar el comando ejecutado en un historial.

Esta funcionalidad convierte la aplicación en una interfaz sencilla para la ejecución de comandos Linux.

8.4. Entrada y salida

La interacción con el usuario se implementa mediante las funciones estándar de entrada y salida del lenguaje C.

Las principales funciones utilizadas son:

- `printf()` para mostrar información.
- `scanf()` para capturar datos ingresados por el usuario.
- `fgets()` para leer cadenas de texto completas.
- Funciones de manejo de archivos como `fopen()`, `fprintf()`, `fclose()` y `fgets()` para almacenar historiales, respaldos y registros.

8.5. Manejo de directorios

Para recorrer directorios y acceder a su contenido se emplean funciones proporcionadas por la biblioteca POSIX.

Entre ellas destacan:

- `opendir()`
- `readdir()`
- `closedir()`

Estas funciones permiten listar archivos y obtener información del contenido de un directorio determinado.

8.6. Resumen

La combinación de funciones estándar del lenguaje C, bibliotecas POSIX y comandos propios de Linux permitió implementar una herramienta funcional para la administración del sistema, manteniendo una estructura modular y de fácil mantenimiento.

9. Compilación e instalación del proyecto

El proyecto fue desarrollado para ejecutarse en sistemas operativos Linux, utilizando el compilador GCC y un archivo `Makefile` que automatiza el proceso de compilación.

9.1. Requisitos del sistema

Para compilar y ejecutar la aplicación se requiere:

- Sistema operativo Linux (Ubuntu 22.04 o superior).
- Compilador GCC.
- Utilidad `make`.
- Permisos de lectura y escritura sobre los directorios del proyecto.

Antes de compilar, es recomendable verificar que GCC y Make se encuentren instalados mediante los siguientes comandos:

```
gcc --version
make --version
```

9.2. Estructura del proyecto

El código fuente se encuentra organizado en diferentes directorios para facilitar su mantenimiento.

- `src/`: archivos fuente (.c).
- `include/`: archivos de cabecera (.h).
- `bin/`: ejecutable generado.
- `backups/`: almacenamiento de respaldos.
- `downloads/`: archivos relacionados con las descargas.
- `logs/`: registros de eventos e historial.
- `scripts/`: scripts utilizados para pruebas del analizador Bash.

9.3. Proceso de compilación

La compilación completa del proyecto se realiza ejecutando el siguiente comando desde la carpeta principal del repositorio:

```
make
```

El archivo Makefile compila todos los módulos del sistema y genera el ejecutable principal dentro del directorio correspondiente.

9.4. Limpieza del proyecto

Para eliminar los archivos generados durante la compilación se utiliza el siguiente comando:

```
make clean
```

Esta operación elimina los archivos objeto y el ejecutable generado, permitiendo realizar una compilación limpia cuando sea necesario.

9.5. Ejecución del programa

Una vez compilado el proyecto, la aplicación puede ejecutarse desde la terminal mediante:

```
./bin/admin_linux
```

Al iniciar, el programa muestra el menú principal desde el cual el usuario puede acceder a todas las funcionalidades implementadas.

10. Pruebas realizadas

Con el objetivo de verificar el correcto funcionamiento de la herramienta, se realizaron diferentes pruebas sobre cada uno de los módulos implementados. Estas pruebas permitieron comprobar que las funcionalidades principales operan de acuerdo con los requisitos establecidos para el proyecto.

10.1. Pruebas del administrador de tareas

Se verificó el funcionamiento de las operaciones relacionadas con la administración de procesos del sistema.

Las pruebas realizadas fueron:

- Listado de procesos activos.

```
sharon@LAPTOP-86PMU6II: /mn x Sharon@LAPTOP-86PMU6II: ~ x + v
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspendar proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
0. Volver al menú principal
=====
Seleccione una opción: 1

--- Listando procesos (ps aux) ---
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1   0.0  0.3 167624 12944 ?        Ss   18:09   0:03 /sbin/init
root         2   0.0  0.0   2776  1920 ?        Sl   18:09   0:00 /init
root         6   0.0  0.0   2776   132 ?        Sl   18:09   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd
root        64   0.0  0.4  64276 15392 ?        S<s  18:09   0:01 /lib/systemd/systemd-journald
root        93   0.0  0.1  22748  6652 ?        Ss   18:09   0:01 /lib/systemd/systemd-udev
root       104   0.0  0.0 152992   160 ?        Ssl  18:09   0:00 snapfuse /var/lib/snapd/snaps/core20_2769.snap /snap/core20/276
root       106   0.0  0.3 452048 12460 ?        Ssl  18:09   0:00 snapfuse /var/lib/snapd/snaps/core20_2866.snap /snap/core20/286
root       115   0.0  0.0 152992  2256 ?        Ssl  18:09   0:00 snapfuse /var/lib/snapd/snaps/lxd_38688.snap /snap/lxd/38688 -o
root       116   0.0  0.0 153124  2504 ?        Ssl  18:09   0:00 snapfuse /var/lib/snapd/snaps/lxd_40111.snap /snap/lxd/40111 -o
root       126   0.0  0.5 601576 20436 ?        Ssl  18:09   0:05 snapfuse /var/lib/snapd/snaps/snapd_27406.snap /snap/snapd/2740
systemd+   137   0.0  0.1  89364  6532 ?        Ssl  18:09   0:00 /lib/systemd/systemd-timesyncd
systemd+   144   0.0  0.2  16132   7828 ?        Ss   18:09   0:00 /lib/systemd/systemd-networkd
message+   282   0.0  0.1   8756  4632 ?        Ss   18:09   0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile -
root       285   0.0  0.5 33188 19036 ?        Ss   18:09   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-tri
root       286   0.0  0.2 234528  8692 ?        Ssl  18:09   0:00 /usr/libexec/polkitd --no-debug
root       287   0.0  0.1 222484  7036 ?        Ssl  18:09   0:00 /usr/sbin/rsyslogd -n -iNONE
root       294   0.0  0.2  15728  8140 ?        Ss   18:09   0:00 /lib/systemd/systemd-logind
root       299   0.0  0.3 25548 12472 ?        Ss   18:09   0:00 /lib/systemd/systemd-resolved
root       309   0.0  0.3 392704 12760 ?        Ssl  18:09   0:00 /usr/libexec/udisks2/udisksd
redis       372   0.1  0.4  67244 16932 ?        Ssl  18:09   0:10 /usr/bin/redis-server 127.0.0.1:6379
root       374   0.0  0.5 110156 21704 ?        Ssl  18:09   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgr
root       375   0.0  1.3 2171412 51780 ?        Ssl  18:09   0:05 /usr/bin/containerd
mysql       470   0.0  3.5 1296924 131252 ?        Ssl  18:09   0:02 /usr/sbin/mariadb
root       775   0.0  0.0   2776   204 ?        Ss   18:09   0:00 /init
root       776   0.0  0.0   2776  1152 ?        S   18:09   0:00 /init
sharon     783   0.0  0.1   9312  5284 pts/0    Ss   18:09   0:00 -bash
root     1119   0.0  0.3 302520 13340 ?        Ssl  18:09   0:01 snapfuse /var/lib/snapd/snaps/snapd_27591.snap /snap/snapd/2759
root     1129   0.0  1.8 2591128 69728 ?        Ssl  18:09   0:03 /snap/snapd/current/usr/lib/snapd/snapd
root     1350   0.0  2.6 2574668 109480 ?        Ssl  18:11   0:03 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containe
root     1354   0.0  0.0   7288   2768 ?        Ss   18:11   0:00 /usr/sbin/cron -f -P
root     1371   0.0  0.0   6228  1076 hvc0    Ss+  18:11   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud console 115200,3
root     1380   0.0  0.0   6176  1148 tty1    Ss+  18:11   0:00 /sbin/agetty -o -p -- \u --noclear tty1 linux
root     1383   0.0  0.1  6860  4700 ?        Ss   18:11   0:00 /usr/sbin/apache2 -k start
www-data   1384   0.0  0.1  6828  4200 ?        S   18:11   0:00 /usr/sbin/apache2 -k start
```

Figura 1: Listado de procesos activos.

- Búsqueda de procesos por nombre.

```
=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspender proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
0. Volver al menú principal
=====
Seleccione una opción: 2
Ingrese el nombre del proceso a buscar: 167624

--- Buscando procesos con '167624' ---
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.3 167624 12944 ?        Ss   18:09   0:03 /sbin/init
sharon    3167  0.0  0.0   2892   932 pts/0    S+   19:57   0:00 sh -c ps aux | grep -E --color=never 'USER|167624'
sharon    3169  0.0  0.0   7004  2040 pts/0    S+   19:57   0:00 grep -E --color=never USER|167624
```

Figura 2: Búsqueda de procesos por nombre.

- Visualización del consumo de CPU y memoria.

```
=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspender proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
0. Volver al menú principal
=====
Seleccione una opción: 3

--- Consumo de CPU y memoria (top 5 procesos) ---
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
redis     372  0.1  0.4  67244 16932 ?        Ssl  18:09   0:11 /usr/bin/redis-server 127.0.0.1:6379
root         1  0.0  0.3 167624 12944 ?        Ss   18:09   0:03 /sbin/init
root         2  0.0  0.0   2776  1920 ?        Sl   18:09   0:00 /init
root         6  0.0  0.0   2776   132 ?        Sl   18:09   0:00 plan9 --control-socket 7 --log-level 4 --server-fd 8 --pipe-fd 10 --log-trunc
ate
root        64  0.0  0.4  64276 15404 ?        S<s  18:09   0:01 /lib/systemd/systemd-journald

=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
```

Figura 3: Visualización del consumo de CPU y memoria.

- Finalización de procesos seleccionados.

```
fabiana@DESKTOP-E6NUN7: ~$ ps aux
--- Listando procesos (ps aux) ---
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 21784 12960 ?        Ss   18:55   0:01 /sbin/init
root         2  0.0  0.0  2476  1432 ?        Ss   18:55   0:00 /init
root         7  0.0  0.0   2492   132 ?        Ss   18:55   0:00 plan9 --control-socket 6 --log-level 4 --server-fd 7 --pipe-fd 9 --log-truncate
root        58  0.0  0.2 66044 19748 ?        S<Ss 18:55   0:00 /usr/lib/systemd/systemd-journald
root        99  0.0  0.0 24148  6308 ?        Ss   18:55   0:01 /usr/lib/systemd/systemd-udev
systemd+  110  0.0  0.1 21336 11760 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd-resolved
systemd+  118  0.0  0.0 91028  6596 ?        Ssl  18:55   0:00 /usr/lib/systemd/systemd-timesyncd
root       174  0.0  0.0   4236  2652 ?        Ss   18:55   0:00 /usr/sbin/cron -f -P
message+  175  0.0  0.0   9624  5240 ?        Ss   18:55   0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-only
root       199  0.0  0.1 17984  8184 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd-logind
root       203  0.0  0.2 1756096 17984 ?       Ssl  18:55   0:00 /usr/libexec/wsl-pro-service -vv
root       216  0.0  0.0   3160  1160 hvcd ?        Ss+  18:55   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root       229  0.0  0.0   3116  1148 tty1 ?        Ss+  18:55   0:00 /sbin/agetty -o -p -- \u --noclear - linux
syslog    231  0.0  0.0 222508   536 ?        Ssl  18:55   0:00 /usr/sbin/rsyslogd -n -iNONE
root       241  0.1  0.8 2237904 67456 ?       Ssl  18:55   0:00 /usr/bin/containerd
root       242  0.0  0.2 107028 22604 ?       Ssl  18:55   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root       289  0.0  0.0   6880  4892 ?        Ss   18:55   0:00 /usr/sbin/apache2 -k start
www-data  290  0.0  0.0   7892  4680 ?        Ss   18:55   0:00 /usr/sbin/apache2 -k start
www-data  292  0.0  0.1 1999388 13428 ?       Sl   18:55   0:00 /usr/sbin/apache2 -k start
www-data  293  0.0  0.1 1999388 13428 ?       Sl   18:55   0:00 /usr/sbin/apache2 -k start
mysql     379  1.0  5.2 2376244 420196 ?       Ssl  18:55   0:46 /usr/sbin/mysqld
root      501  0.0  1.1 2427384 93216 ?       Ssl  18:55   0:01 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root      583  0.0  0.0   6692  4668 pts/1 ?        Ss   18:55   0:00 /bin/login -f
fabiana   636  0.0  0.1 20272 11524 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd --user
fabiana   638  0.0  0.0  21152  1724 ?        Ss   18:55   0:00 (sd-pam)
fabiana   663  0.0  0.1 10412  9264 pts/1 ?        S+   18:55   0:00 -bash
polkitd   1784  0.0  0.1 308164  9220 ?       Ssl  18:58   0:00 /usr/lib/polkit-1/polkitd --no-debug
root     2511  0.0  0.0   2492   116 ?        Ss   19:01   0:00 /init
root     2512  0.0  0.0   2492   124 ?        Ss   19:01   0:00 /init
fabiana   2513  0.0  0.1 10548  9708 pts/2 ?        Ss   19:01   0:00 -bash
fabiana  3323  0.0  0.0   2708  1224 pts/2 ?        T    20:09   0:00 ./bin/admin_linux
fabiana  3338  0.0  0.0   2708  1096 pts/2 ?        S+   20:12   0:00 ./bin/admin_linux
fabiana  3341 150  0.0   8336  4304 pts/2 ?        R+   20:12   0:00 ps aux

=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspender proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
8. Volver al menú principal
=====
Seleccione una opción: 4
Ingrese el PID del proceso a finalizar: 3323
Proceso 3323 finalizado correctamente.
```

Figura 4: Finalización de un proceso mediante su PID.

- Suspensión y reanudación de procesos.

```
fabiana@DESKTOP-E6NUN7: ~$ ps aux
--- Listando procesos (ps aux) ---
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1 21784 12960 ?        Ss   18:55   0:01 /sbin/init
root         2  0.0  0.0  2476  1432 ?        Ss   18:55   0:00 /init
root         7  0.0  0.0   2492   132 ?        Ss   18:55   0:00 plan9 --control-socket 6 --log-level 4 --server-fd 7 --pipe-fd 9 --log-t
root        58  0.0  0.2 66044 19748 ?        S<Ss 18:55   0:00 /usr/lib/systemd/systemd-journald
root        99  0.0  0.0 24148  6308 ?        Ss   18:55   0:01 /usr/lib/systemd/systemd-udev
systemd+  110  0.0  0.1 21336 11760 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd-resolved
systemd+  118  0.0  0.0 91028  6596 ?        Ssl  18:55   0:00 /usr/lib/systemd/systemd-timesyncd
root       174  0.0  0.0   4236  2652 ?        Ss   18:55   0:00 /usr/sbin/cron -f -P
message+  175  0.0  0.0   9624  5240 ?        Ss   18:55   0:00 dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-
root       199  0.0  0.1 17984  8184 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd-logind
root       203  0.0  0.2 1756096 17984 ?       Ssl  18:55   0:00 /usr/libexec/wsl-pro-service -vv
root       216  0.0  0.0   3160  1160 hvcd ?        Ss+  18:55   0:00 /sbin/agetty -o -p -- \u --noclear --keep-baud - 115200,38400,9600 vt220
root       229  0.0  0.0   3116  1148 tty1 ?        Ss+  18:55   0:00 /sbin/agetty -o -p -- \u --noclear - linux
syslog    231  0.0  0.0 222508   536 ?        Ssl  18:55   0:00 /usr/sbin/rsyslogd -n -iNONE
root       241  0.1  0.8 2237904 67456 ?       Ssl  18:55   0:00 /usr/bin/containerd
root       242  0.0  0.2 107028 22604 ?       Ssl  18:55   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutd
root       289  0.0  0.0   6880  4892 ?        Ss   18:55   0:00 /usr/sbin/apache2 -k start
www-data  290  0.0  0.0   7892  4680 ?        Ss   18:55   0:00 /usr/sbin/apache2 -k start
www-data  292  0.0  0.1 1999388 13428 ?       Sl   18:55   0:00 /usr/sbin/apache2 -k start
www-data  293  0.0  0.1 1999388 13428 ?       Sl   18:55   0:00 /usr/sbin/apache2 -k start
mysql     379  1.0  5.2 2376244 420196 ?       Ssl  18:55   0:45 /usr/sbin/mysqld
root      501  0.0  1.1 2427384 93216 ?       Ssl  18:55   0:01 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
root      583  0.0  0.0   6692  4668 pts/1 ?        Ss   18:55   0:00 /bin/login -f
fabiana   636  0.0  0.1 20272 11524 ?        Ss   18:55   0:00 /usr/lib/systemd/systemd --user
fabiana   638  0.0  0.0  21152  1724 ?        Ss   18:55   0:00 (sd-pam)
fabiana   663  0.0  0.1 10412  9264 pts/1 ?        S+   18:55   0:00 -bash
polkitd   1784  0.0  0.1 308164  9220 ?       Ssl  18:58   0:00 /usr/lib/polkit-1/polkitd --no-debug
root     2511  0.0  0.0   2492   116 ?        Ss   19:01   0:00 /init
root     2512  0.0  0.0   2492   124 ?        Ss   19:01   0:00 /init
fabiana   2513  0.0  0.1 10548  9708 pts/2 ?        Ss   19:01   0:00 -bash
fabiana  3323  0.0  0.0   2708  1224 pts/2 ?        T    20:09   0:00 ./bin/admin_linux
fabiana  3324  0.0  0.0   8336  4348 pts/2 ?        R+   20:09   0:00 ps aux

=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspender proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
8. Volver al menú principal
=====
Seleccione una opción: 5
Ingrese el PID del proceso a suspender: 3323

[1]+  Stopped                  ./bin/admin_linux
fabiana@DESKTOP-E6NUN7C:/mnt/d/2026A/PROGRAMACION DE SISTEMAS/prueba/Programaci-n-de-Sistemas---PROYECTO-FINAL-$
```

Figura 5: Suspensión de un proceso mediante la señal SIGSTOP.

- Visualización del árbol de procesos.

```
fabiana@DESKTOP-E6NUN: ~$   
=====   
ADMINISTRADOR DE PROCESOS   
=====   
1. Listar procesos (ps aux)   
2. Buscar proceso por nombre   
3. Mostrar consumo de CPU y memoria   
4. Finalizar proceso (kill)   
5. Suspender proceso (kill -STOP)   
6. Reanudar proceso (kill -CONT)   
7. Mostrar árbol de procesos (pstree)   
8. Volver al menú principal   
=====   
Seleccione una opción: 7   
--- Árbol de procesos (pstree -p) ---   
systemd(1)---agetty(216)   
            | agetty(229)   
            |---apache2(289)   
                |---apache2(290)   
                    |---apache2(292)   
                        {apache2}(296)   
                        {apache2}(299)   
                        {apache2}(301)   
                        {apache2}(303)   
                        {apache2}(305)   
                        {apache2}(306)   
                        {apache2}(307)   
                        {apache2}(308)   
                        {apache2}(310)   
                        {apache2}(314)   
                        {apache2}(316)   
                        {apache2}(319)   
                        {apache2}(320)   
                        {apache2}(321)   
                        {apache2}(323)   
                        {apache2}(324)   
                        {apache2}(328)   
                        {apache2}(329)   
                        {apache2}(331)   
                        {apache2}(332)   
                        {apache2}(333)   
                        {apache2}(335)   
                        {apache2}(337)   
                        {apache2}(342)   
                        {apache2}(343)   
                        {apache2}(344)   
                    |---apache2(293)   
                        {apache2}(297)   
                        {apache2}(298)   
                        {apache2}(300)   
                        {apache2}(302)   
                        {apache2}(304)   
                        {apache2}(309)   
                        {apache2}(311)   
                        {apache2}(312)
```

Figura 6: Árbol de procesos del sistema.

Los resultados obtenidos demostraron que el módulo permite consultar y administrar procesos del sistema de manera correcta.

10.2. Pruebas del shell de archivos

Se realizaron pruebas utilizando archivos y directorios de ejemplo.

- Listado del contenido de directorios.


```

www-data 1385 0.0 0.3 1998260 12676 ? SL 18:12 0:00 /usr/sbin/apache2 -k start
www-data 1386 0.0 0.3 1998260 12676 ? SL 18:12 0:00 /usr/sbin/apache2 -k start
root 2156 0.0 0.0 2776 208 ? Ss 18:39 0:00 /init
root 2157 0.0 0.0 2776 208 ? S 18:39 0:00 /init
sharon 2162 0.0 0.1 9388 5252 pts/2 Ss 18:39 0:00 -bash
root 2542 0.0 0.5 296024 20220 ? Ssl 18:40 0:00 /usr/libexec/packagekitd
root 2717 0.0 0.1 11060 4592 pts/2 S 18:44 0:00 su - sharon
sharon 2719 0.0 0.2 17068 9320 ? Ss 18:44 0:00 /lib/systemd/systemd --user
sharon 2720 0.0 0.1 170408 4932 ? S 18:44 0:00 (sd-pam)
sharon 2726 0.0 0.1 9232 5244 pts/2 S+ 18:44 0:00 -bash
sharon 3160 0.0 0.0 2800 1092 pts/0 S+ 19:52 0:00 ./bin/admin_linux
root 3193 1.8 0.4 1756716 16180 ? Ssl 20:14 0:00 /usr/libexec/wsl-pro-service
sharon 3216 0.0 0.0 10464 3272 pts/0 R+ 20:14 0:00 ps aux

=====
ADMINISTRADOR DE PROCESOS
=====
1. Listar procesos (ps aux)
2. Buscar proceso por nombre
3. Mostrar consumo de CPU y memoria
4. Finalizar proceso (kill)
5. Suspender proceso (kill -STOP)
6. Reanudar proceso (kill -CONT)
7. Mostrar árbol de procesos (pstree)
8. Volver al menú principal
=====
Seleccione una opción: 4
Ingrese el PID del proceso a finalizar: 286

```

Figura 7: Listado del contenido de un directorio.

■ Copia de archivos.

```

=====
SHELL DE ARCHIVOS
=====
1. Listar archivos
2. Copiar / Mover archivos
3. Eliminar archivo
4. Buscar archivo
5. Mostrar estadísticas
0. Regresar
=====
Seleccione una opción: 2

===== COPIAR / MOVER =====
1. Copiar archivo
2. Mover archivo
Seleccione una opción: README.md
Ruta del archivo origen: Ruta destino:
docs

```

Figura 8: Copia de un archivo hacia un nuevo destino.

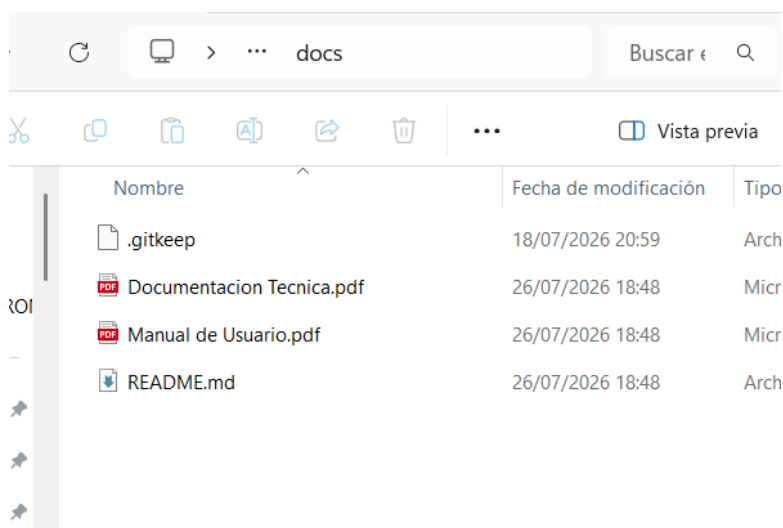


Figura 9: En docs se copio el archivo README.md.

■ Eliminación de archivos.

```
=====
SHELL DE ARCHIVOS
=====
1. Listar archivos
2. Copiar / Mover archivos
3. Eliminar archivo
4. Buscar archivo
5. Mostrar estadísticas
0. Regresar
=====
Seleccione una opcion: 3

===== ELIMINAR ARCHIVO =====
Nombre del archivo: README.md
Archivo eliminado correctamente.
```

Figura 10: Eliminación de un archivo.

■ **Búsqueda de archivos por nombre.**

```
=====
SHELL DE ARCHIVOS
=====
1. Listar archivos
2. Copiar / Mover archivos
3. Eliminar archivo
4. Buscar archivo
5. Mostrar estadísticas
0. Regresar
=====
Seleccione una opcion: 4

===== BUSCAR ARCHIVO =====
Nombre del archivo: Makefile
Archivo encontrado: Makefile
```

Figura 11: Búsqueda de archivos mediante su nombre.

■ **Consulta de estadísticas de archivos.**

```
=====
SHELL DE ARCHIVOS
=====
1. Listar archivos
2. Copiar / Mover archivos
3. Eliminar archivo
4. Buscar archivo
5. Mostrar estadísticas
0. Regresar
=====
Seleccione una opcion: 5

===== ESTADISTICAS =====
Nombre del archivo: Makefile

Informacion del archivo
-----
Tamano: 360 bytes
Numero de enlaces: 1
UID propietario: 1000
GID propietario: 1000
Permisos (octal): 777
```

Figura 12: Consulta de la información y estadísticas de un archivo.

ulta de estadísticas de archivos.

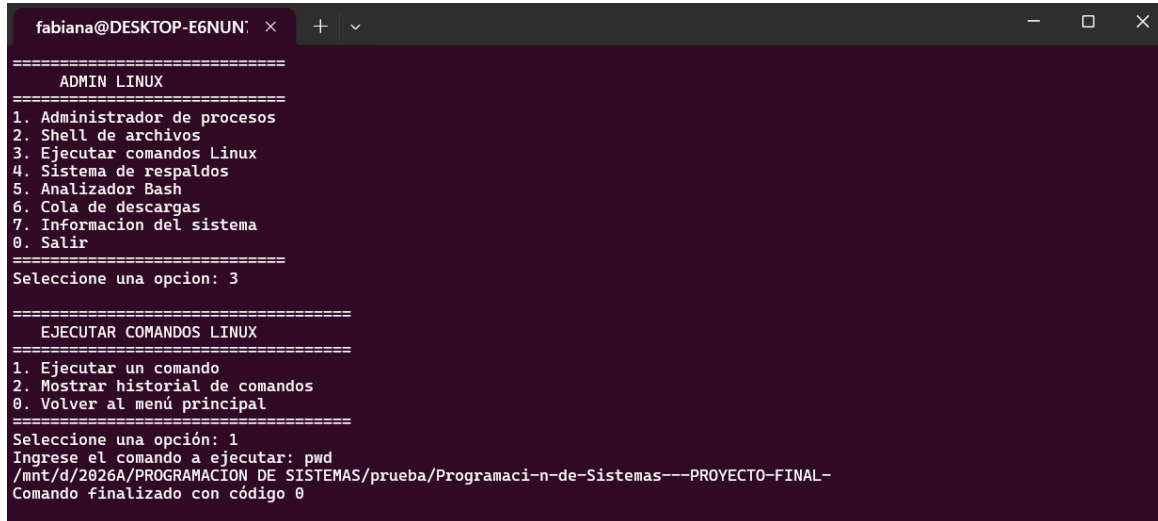
Todas las operaciones produjeron los resultados esperados bajo condiciones normales de uso.

10.3. Pruebas de ejecución de comandos

Se ejecutaron distintos comandos propios de Linux para verificar la correcta comunicación entre la aplicación y el sistema operativo.

Entre los comandos probados se encuentran:

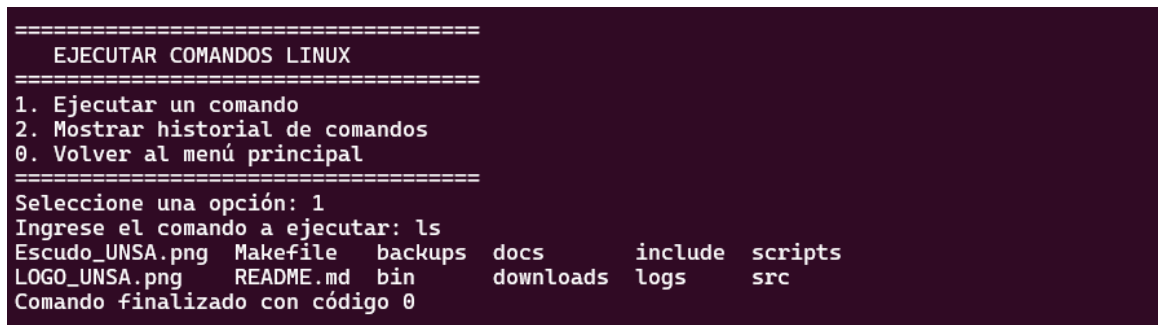
- pwd



```
fabiana@DESKTOP-E6NUN: x + v
=====
ADMIN LINUX
=====
1. Administrador de procesos
2. Shell de archivos
3. Ejecutar comandos Linux
4. Sistema de respaldos
5. Analizador Bash
6. Cola de descargas
7. Informacion del sistema
0. Salir
=====
Seleccione una opcion: 3
=====
EJECUTAR COMANDOS LINUX
=====
1. Ejecutar un comando
2. Mostrar historial de comandos
0. Volver al menú principal
=====
Seleccione una opción: 1
Ingrese el comando a ejecutar: pwd
/mnt/d/2026A/PROGRAMACION DE SISTEMAS/prueba/Programaci-n-de-Sistemas---PROYECTO-FINAL-
Comando finalizado con código 0
```

Figura 13: Visualización de la ruta del directorio de trabajo actual.

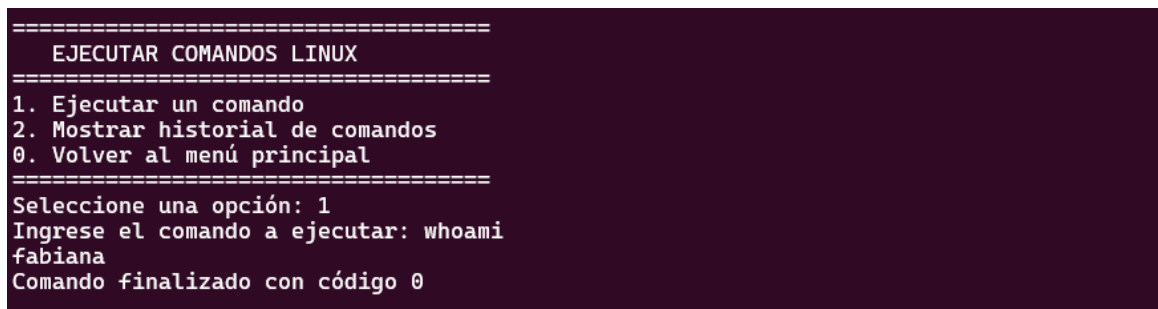
- ls



```
=====
EJECUTAR COMANDOS LINUX
=====
1. Ejecutar un comando
2. Mostrar historial de comandos
0. Volver al menú principal
=====
Seleccione una opción: 1
Ingrese el comando a ejecutar: ls
Escudo_UNSA.png Makefile backups docs include scripts
LOGO_UNSA.png README.md bin downloads logs src
Comando finalizado con código 0
```

Figura 14: Listado del contenido y archivos del directorio actual.

- whoami



```
=====
EJECUTAR COMANDOS LINUX
=====
1. Ejecutar un comando
2. Mostrar historial de comandos
0. Volver al menú principal
=====
Seleccione una opción: 1
Ingrese el comando a ejecutar: whoami
fabiana
Comando finalizado con código 0
```

Figura 15: Identificación del usuario activo en la sesión del Shell.

- `date`

```
=====
EJECUTAR COMANDOS LINUX
=====
1. Ejecutar un comando
2. Mostrar historial de comandos
0. Volver al menú principal
=====
Seleccione una opción: 1
Ingrese el comando a ejecutar: date
Sun Jul 26 20:21:41 -05 2026
Comando finalizado con código 0
```

Figura 16: Despliegue de la fecha y hora actual del sistema.

Además, se comprobó que el historial de comandos registra correctamente las ejecuciones realizadas.

10.4. Pruebas del sistema de respaldos

Se verificó la creación y restauración de respaldos utilizando archivos de prueba. Las pruebas permitieron comprobar:

- Creación de nuevas copias de seguridad.
- Almacenamiento organizado de respaldos.
- Restauración de versiones previamente almacenadas.

Los archivos restaurados conservaron correctamente su contenido original.

10.5. Pruebas del analizador Bash

Para validar este módulo se utilizaron diferentes scripts Bash con estructuras variadas. Durante las pruebas se verificó la detección de:

- Variables declaradas.
- Ciclos `for`.
- Ciclos `while`.

Los resultados obtenidos coincidieron con la estructura de los scripts analizados.

10.6. Pruebas de la cola de descargas

Se realizaron pruebas agregando diferentes elementos a la cola de descargas. Se verificó:

- Registro correcto de nuevas tareas.
- Visualización de la cola.
- Registro de eventos.
- Actualización del progreso de las descargas.

El módulo respondió correctamente durante todas las pruebas realizadas.

10.7. Pruebas del módulo de Información del Sistema

Se realizaron pruebas ejecutando las diferentes opciones del módulo de información del sistema para verificar la correcta obtención de datos del equipo mediante comandos Linux.

Se verificó:

- Obtención correcta de la información del sistema operativo.
- Visualización de las características del procesador.
- Consulta adecuada del estado de la memoria RAM.
- Visualización del espacio disponible en disco.
- Obtención del tiempo de actividad del sistema.
- Identificación correcta del usuario actual.

10.8. Resumen de pruebas

En la Tabla 1 se presenta un resumen de las pruebas ejecutadas sobre cada uno de los módulos del sistema.

Módulo	Resultado
Administrador de tareas	Correcto
Shell de archivos	Correcto
Comandos Linux	Correcto
RespalDOS	Correcto
Analizador Bash	Correcto
Cola de descargas	Correcto

Cuadro 1: Resumen de las pruebas funcionales realizadas.

11. Conclusiones y trabajos futuros

11.1. Conclusiones

El desarrollo de la herramienta permitió integrar en una única aplicación diversas funcionalidades de administración para sistemas Linux, cumpliendo con los objetivos planteados al inicio del proyecto. La estructura modular implementada facilitó la organización del código y permitió desarrollar cada componente de forma independiente, favoreciendo el mantenimiento y la escalabilidad del sistema.

Durante la implementación se aplicaron conceptos propios de la programación de sistemas, haciendo uso del lenguaje C, bibliotecas estándar y herramientas del sistema operativo Linux para la gestión de procesos, archivos, ejecución de comandos y administración de recursos.

Cada uno de los módulos desarrollados fue probado de manera independiente, verificando el correcto funcionamiento de las operaciones de administración de procesos, manipulación de archivos, ejecución de comandos, creación y restauración de respaldos, análisis de scripts Bash y gestión de la cola de descargas.

Asimismo, el uso de un archivo Makefile permitió automatizar el proceso de compilación, mientras que la organización del proyecto en módulos independientes contribuyó a mejorar la legibilidad y facilitar futuras modificaciones del código.

En conjunto, el proyecto permitió aplicar de forma práctica los conocimientos adquiridos durante el curso de Programación de Sistemas, integrando diferentes conceptos del sistema operativo Linux en una aplicación funcional.

11.2. Trabajos futuros

Como parte de una posible evolución del proyecto, se plantean las siguientes mejoras:

- Implementar una interfaz gráfica utilizando bibliotecas como GTK o Qt.
- Incorporar monitoreo de procesos en tiempo real con actualización automática de la información.
- Mejorar el sistema de respaldos mediante compresión y programación automática de copias de seguridad.
- Ampliar el analizador de scripts Bash para detectar funciones, condiciones, tuberías y llamadas a otros scripts.
- Agregar soporte para múltiples usuarios y distintos niveles de permisos.
- Implementar un sistema de configuración para personalizar rutas, directorios y preferencias de la aplicación.
- Optimizar el manejo de errores y la validación de entradas para incrementar la robustez del sistema.

Estas mejoras permitirían ampliar las capacidades de la herramienta y acercarla al funcionamiento de aplicaciones de administración utilizadas en entornos Linux reales.